

UNEB — Universidade do Estado da Bahia
Tópicos Especiais em Engenharia de Software
Prof. Eduardo Manuel de Freitas Jorge

Projeto Arquitetural

Sistema de Busca de Informações de Pesquisadores Baseado em Full-Text Search e LLM

Versão 1.0 — Sprint I
Maio de 2026

1. Visão Arquitetural

O sistema adota uma arquitetura em camadas, onde cada componente tem responsabilidade bem definida e se comunica com os demais por interfaces padronizadas. A separação em camadas garante que mudanças em uma camada não impactem diretamente as demais, facilitando manutenção, testes e evolução do produto.

A solução é composta por cinco camadas principais:

- Camada de Dados Brutos: arquivos XML do Currículo Lattes como fonte de entrada
- Camada de Orquestração ETL: Apache Hop processa e carrega os dados no banco
- Camada de Persistência: PostgreSQL com pgvector para dados relacionais e vetoriais
- Camada de Back-End: Flask (Python) expõe API REST com Full-Text Search e busca semântica via LangChain
- Camada de Front-End: interface em HTML, CSS e JavaScript consumindo a API REST
- Camada Analítica: geração de CSVs para Power BI (opcional na v1.0)

2. Diagrama de Componentes (Descrição Textual)

O fluxo de dados segue a sequência abaixo:



3. Detalhamento por Camada

3.1 Camada de Dados Brutos

FONTE DE DADOS — Currículo Lattes XML

- Formato: XML padronizado pelo CNPq
- Conteúdo: dados pessoais, formação, produções científicas, áreas de atuação
- Local: diretório de entrada configurável no pipeline Apache Hop
- Processamento: individual por arquivo; um arquivo por pesquisador

3.2 Camada de Orquestração ETL

APACHE HOP — Pipeline ETL

- Responsabilidade: extrair, transformar e carregar dados do Lattes para o PostgreSQL
- Operações: leitura de XML, mapeamento de campos, limpeza, deduplicação e INSERT/UPSERT
- Componentes do pipeline: Input XML → Transform → Lookup (dedup) → Output PostgreSQL
- Execução: manual via interface Hop ou agendada via script de linha de comando
- Log: cada execução gera registro com status, erros e quantidade de registros processados

3.3 Camada de Persistência

POSTGRESQL + PGVECTOR

- Banco relacional: armazena pesquisadores, produções, áreas e palavras-chave
- Full-Text Search: índice TSVECTOR na coluna título das produções (pt/unaccent)
- pgvector: extensão para armazenamento e consulta de embeddings (VECTOR 1536)

- Índice vetorial: IVFFlat ou HNSW para busca por similaridade de cosseno
- Gerenciamento: HeidiSQL ou dBeaver ou psql para administração e consultas de suporte

3.4 Camada de Back-End

FLASK (PYTHON) + LANGCHAIN
• Framework: Flask com estrutura modular (blueprints por domínio) • Full-Text Search: endpoint /busca?q={termo} usa to_tsquery e ts_rank do PostgreSQL • Busca Semântica: endpoint /busca-semantic usa LangChain para gerar embedding da query • pgvector: consulta de K-vizinhos mais próximos por distância de cosseno • Exportação CSV: endpoint /exportar/csv gera dimensões e fato para Power BI • Documentação: página HTML dedicada com todos os endpoints, parâmetros e exemplos • Containerização: Dockerfile para execução isolada e consistente

3.5 Camada de Front-End

HTML + CSS + JAVASCRIPT (Vanilla)
• Tecnologias: HTML5 semântico, CSS3 (Flexbox/Grid), JavaScript ES6+ • Telas principais: Home/Busca, Resultados, Perfil do Pesquisador, Exportação • Comunicação: Fetch API para chamadas assíncronas à API Flask • Responsividade: layout adaptável para desktop e tablet • UX: indicadores de loading, mensagens de erro e paginação de resultados

3.6 Camada Analítica

POWER BI — Análises e Dashboards
• Entrada: arquivos CSV gerados pelo back-end Flask • Modelo estrela: dimensões Pesquisador, Tempo (Ano/Quadriênio) e Fato Produção • Métricas esperadas: total de produções por pesquisador, área e período quadrienal • Distribuição: arquivos CSV disponibilizados via download no front-end

4. Stack Tecnológica

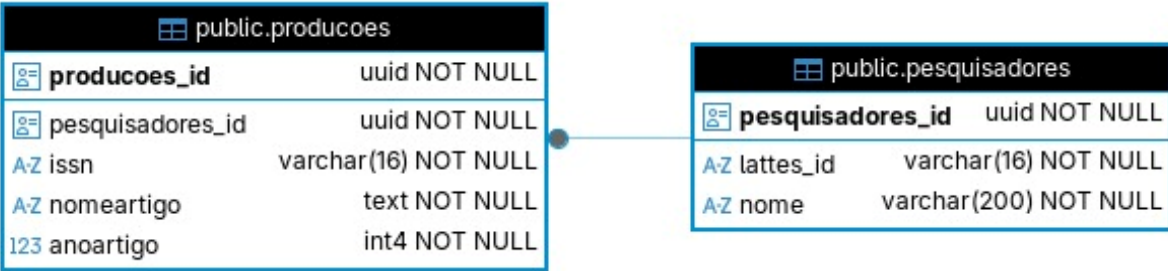
Camada	Tecnologia	Justificativa
Controle de Versão	Git / GitHub	Rastreabilidade, histórico e colaboração
Containerização	Docker / Docker Compose	Consistência de ambiente entre dev e produção
ETL	Apache Hop	Interface visual para pipelines de dados, sem

		código
Banco de Dados	PostgreSQL 15+	Robusto, suporta FTS nativo e extensão pgvector
Vetorial	pgvector	Armazenamento e busca de embeddings no próprio PostgreSQL
Back-End	Python 3.11 + Flask	Microframework leve, rápido de configurar, ampla documentação
IA / Embeddings	LangChain + OpenAI / HuggingFace	Abstração de LLMs; geração de embeddings padronizada
Front-End	HTML5 + CSS3 + JS Vanilla	Sem dependências de framework; simples e portátil
Administração DB	HeidiSQL ou dBeaver	Interface gráfica para PostgreSQL
Análise / BI	Power BI Desktop	Dashboards e relatórios gerenciais sobre os CSVs
Editor de Código	Visual Studio Code	Extensões para Python, Git e Docker integradas

5. Modelo Entidade-Relacionamento (Conceitual)

5.1 Entidades Principais e Relacionamentos

O modelo de dados é centrado na entidade Pesquisador, que se relaciona com Producao_Cientifica (N:N pode ser simplificado para 1:N no escopo inicial), Area_Atuario e Palavra_Chave.



6. Fluxos de Dados

6.1 Fluxo de Busca Textual (Full-Text Search)

Passo	Componente	Ação
1	Usuário / Browser	Digita termo e pressiona buscar; JS faz GET /busca?q=termo
2	Flask API	Recebe parâmetro q, chama função de busca textual
3	PostgreSQL	Executa SELECT com to_tsquery e ts_rank sobre índice GIN
4	Flask API	Serializa resultado para JSON e retorna HTTP 200
5	Browser	JavaScript renderiza a lista de produções na tela

6.2 Fluxo de Busca Semântica

Passo	Componente	Ação
1	Usuário / Browser	Seleciona busca semântica; JS faz POST /busca-semantic com JSON {query}
2	Flask API	Extraí campo query do body
3	LangChain + LLM API	Gera embedding vetorial da query (ex.: OpenAI text-embedding-ada-002)
4	PostgreSQL + pgvector	Busca K-vizinhos mais próximos por distância de cosseno no índice HNSW
5	Flask API	Retorna resultados com score de similaridade em JSON
6	Browser	Exibe resultados ordenados por relevância semântica

6.3 Fluxo ETL (Apache Hop)

Passo	Componente	Ação
1	Admin	Coloca arquivos XML no diretório de entrada e executa o pipeline
2	Apache Hop — Step Input	Lê e faz parsing dos arquivos XML do Lattes
3	Apache Hop — Transform	Mapeia campos XML para colunas relacionais; limpa dados nulos
4	Apache Hop — Lookup	Verifica duplicidade por id_lattes antes de inserir
5	PostgreSQL	Recebe INSERT/UPSERT de pesquisadores e produções
6	Script Python (pós-ETL)	Gera embeddings dos títulos novos e atualiza coluna embedding via pgvector

7. Estrutura Provisória de Diretórios do Repositório

```
sistema-pesquisadores/
├── docs/                                # Documentação técnica (requisitos, arquitetura,
API)
├── modelagem/                          # Modelo ER, scripts DDL
├── etl/                                # Pipelines Apache Hop
│   └── pipelines/                      # Arquivos .hop e fluxos de trabalho
├── backend/                            # Aplicação Flask (Python)
│   ├── app.py                         # Entry point Flask
│   ├── routes/                        # Blueprints: busca, pesquisadores, exportacao
│   ├── services/                      # Lógica de negócio, FTS, embeddings, CSV
│   ├── db/                            # Conexão PostgreSQL e queries
│   └── requirements.txt                # Dependências Python
├── frontend/                           # Interface HTML/CSS/JS
│   ├── index.html                     # Página de busca
│   ├── perfil.html                    # Perfil do pesquisador
│   ├── css/                           # Estilos
│   └── js/                             # Scripts (api.js, ui.js)
├── analitica/                          # CSVs analíticos e arquivo .pbix do Power BI
├── referencias/                        # Materiais de referência e tutoriais
├── docker-compose.yml                 # Orquestra PostgreSQL + Flask + Hop
├── README.md                          # Visão geral e instruções de execução
├── .gitignore
└── LICENSE.md
```

8. Decisões Arquiteturais

ID	Decisão	Justificativa	Alternativa Descartada
DA-01	Flask ao invés de FastAPI	Alinhado ao escopo do projeto e materiais de nivelamento; menor curva de aprendizado	FastAPI (requer conhecimento de tipagem assíncrona)
DA-02	HTML/CSS/JS Vanilla no Front	Simplicidade, sem necessidade de build step; fácil de hospedar e demonstrar	Next.js (excesso de complexidade para o escopo)
DA-03	pgvector dentro do PostgreSQL	Elimina dependência de banco vetorial externo; dados relacionais e vetoriais no mesmo servidor	Pinecone / Chroma / Weaviate (serviços externos)
DA-04	LangChain para embeddings	Abstração que permite trocar o modelo de embedding sem alterar o código da API	Chamada direta à API OpenAI (acoplamento alto)
DA-05	Apache Hop para ETL	Interface visual, configurável sem código; requerido pelo professor no escopo do projeto	Script Python puro (menos visual, mais difícil de manter)
DA-	Docker Compose	Orquestra PostgreSQL e Flask em	Instalação local manual

06		containers isolados; reprodutível em qualquer máquina	(inconsistente entre devs)
----	--	---	----------------------------

Histórico de Revisão

Data	Versão	Descrição	Autor
06/05/2026	1.0	Versão inicial — Sprint I	Equipe do Projeto
12/05/2026	1.1	Revisão final — Sprint I	Equipe do Projeto